

Nama : Deswita Khansa Rafifah

NIM : 254107020151

Kelas : TI-1G

LAPORAN PRAKTIKUM 14

13.2 Persiapan

➤ **Kode program class Customer06.java**

```
package Praktikum14;

public class Customer06 {
    public int id;
    public String name;

    public Customer06() {

    }

    public Customer06(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public String toString() {
        return " ID: " + this.id + " Nama: " + this.name;
    }
}
```

➤ **Kode program class Book06.java**

```
package Praktikum14;

public class Book06 {
    public String isbn;
    public String title;

    public Book06() {

    }

    public Book06(String isbn, String title) {
        this.isbn = isbn;
        this.title = title;
    }
}
```

```

    public String toString() {
        return "ISBN: " + this.isbn + " Title: " + this.title;
    }
}

```

13.3 Praktikum - Implementasi ArrayList

➤ Kode program class DemoArrayList06.java

```

package Praktikum14;
import java.util.ArrayList;

public class DemoArrayList06 {
    public static void main(String[] args) {
        ArrayList<Customer06> customers = new ArrayList<>(2);

        Customer06 customer1 = new Customer06(1, "Zakia");
        Customer06 customer2 = new Customer06(5, "Budi");

        customers.add(customer1);
        customers.add(customer2);

        customers.add(new Customer06(4,"Cica"));

        for (Customer06 cust : customers) {
            System.out.println(cust.toString());
        }
    }
}

```

➤ Hasil running

ID: 1 Nama: Zakia ID: 5 Nama: Budi ID: 4 Nama: Cica

Compile dan run kode program, di mana object yang baru ditambahkan? Di awal, di tengah, atau di akhir collection?

Jawab: Setelah di compile dan run, object yang baru ditambahkan berada di akhir collection.

➤ Kode program tambahan class DemoArrayList06.java

```

customers.add(2, new Customer06(100,"Rosa"));

```

➤ **Hasil running**

```
ID: 1 Nama: Zakia  
ID: 5 Nama: Budi  
ID: 100 Nama: Rosa  
ID: 4 Nama: Cica
```

Compile dan run kode program. Index pada ArrayList dimulai dari 0 atau 1?

Jawab: Index pada ArrayList dimulai dari 0.

➤ **Kode program tambahan class DemoArrayList06.java**

```
System.out.println(customers.indexOf(customer2));
```

➤ **Hasil running**

```
1  
ID: 1 Nama: Zakia  
ID: 5 Nama: Budi  
ID: 100 Nama: Rosa  
ID: 4 Nama: Cica
```

➤ **Kode program tambahan class DemoArrayList06.java**

```
Customer06 customer = customers.get(1);  
System.out.println(customer.name);  
customer.name = "Budi Utomo";
```

➤ **Hasil running**

```
1  
Budi  
ID: 1 Nama: Zakia  
ID: 5 Nama: Budi Utomo  
ID: 100 Nama: Rosa  
ID: 4 Nama: Cica
```

➤ **Kode program tambahan class DemoArrayList06.java**

```
ArrayList<Customer06> customers = new ArrayList<>();
```

➤ **Hasil running**

```
1  
Budi  
ID: 1 Nama: Zakia  
ID: 5 Nama: Budi Utomo  
ID: 100 Nama: Rosa  
ID: 4 Nama: Cica
```

Cobalah hapus angka 2 saat instansiasi object customers. Apakah ArrayList dapat diinstansiasi tanpa harus menentukan size di awal?

Jawab: Ya, ArrayList dapat diinstansiasi tanpa harus menentukan size di awal.

➤ **Kode program tambahan class DemoArrayList06.java**

```
ArrayList<Customer06> newCustomers = new ArrayList<>();
newCustomers.add(new Customer06(201, "Della"));
newCustomers.add(new Customer06(202, "Victor"));
newCustomers.add(new Customer06(203, "Sarah"));
customers.addAll(newCustomers);
```

➤ **Hasil running**

```
1
Budi
ID: 1 Nama: Zakia
ID: 5 Nama: Budi Utomo
ID: 100 Nama: Rosa
ID: 4 Nama: Cica
ID: 201 Nama: Della
ID: 202 Nama: Victor
ID: 203 Nama: Sarah
```

13.4 Praktikum - Implementasi Stack

➤ **Kode program class StackDemo06.java**

```
package Praktikum14;
import java.util.Stack;

public class StackDemo06 {
    public static void main(String[] args) {
        Book06 book1 = new Book06("1234", "Dasar Pemrograman");
        Book06 book2 = new Book06("7145", "Hafalan Shalat Delisa");
        Book06 book3 = new Book06("3562", "Muhammad Al-Fatih");

        Stack<Book06> books = new Stack<>();
        books.push(book1);
        books.push(book2);
        books.push(book3);

        Book06 temp = books.peek();

        if (temp != null) {
            System.out.println(temp.toString());
        }

        Book06 temp2 = books.pop();
```

```

        if (temp2 != null) {
            System.out.println(temp.toString());
        }
    }
}

```

➤ **Hasil running**

```

ISBN: 3562 Title: Muhammad Al-Fatih
ISBN: 3562 Title: Muhammad Al-Fatih

```

Mengapa perlu ada pengecekan (temp != null)?

Jawab: Karena objek yang diambil dari stack dapat bernilai null, maka perlu dilakukan pengecekan (temp != null) untuk memastikan bahwa objek tersebut ada sebelum diakses atau dicetak. Pengecekan ini bertujuan untuk mencegah terjadinya error seperti NullPointerException dan membuat program berjalan dengan lebih aman.

➤ **Kode program tambahan class StackDemo06.java**

```

for (Book06 book : books) {
    System.out.println(book.toString());
}

```

➤ **Hasil running**

```

ISBN: 3562 Title: Muhammad Al-Fatih
ISBN: 3562 Title: Muhammad Al-Fatih
ISBN: 1234 Title: Dasar Pemrograman
ISBN: 7145 Title: Hafalan Shalat Delisa

```

➤ **Modifikasi kode program class StackDemo06.java (Pencarian elemen pada stack menggunakan method search())**

```

int posisi = books.search(book2);
System.out.println("Buku ini berada pada posisi" + posisi);

```

➤ **Hasil running**

```

ISBN: 3562 Title: Muhammad Al-Fatih
ISBN: 3562 Title: Muhammad Al-Fatih
ISBN: 1234 Title: Dasar Pemrograman
ISBN: 7145 Title: Hafalan Shalat Delisa
Buku ini berada pada posisi 1

```

13.5 Praktikum - Implementasi TreeSet

➤ **Kode program class TreeSetDemo06.java**

```
package Praktikum14;
import java.util.TreeSet;

public class TreeSetDemo06 {
    public static void main(String[] args) {
        TreeSet<String> fruits = new TreeSet<>();

        fruits.add("Mangga");
        fruits.add("Apel");
        fruits.add("Jeruk");
        fruits.add("Jambu");

        for (String temp : fruits) {
            System.out.println(temp);
        }
    }
}
```

➤ **Hasil running**

Apel Jambu Jeruk Mangga

Compile dan run program. Mengapa urutan yang ditampilkan berbeda dengan urutan penambahan data ke dalam TreeSet fruits?

Jawab: Karena TreeSet secara otomatis mengurutkan elemen berdasarkan urutan natural (ascending) dan tidak menyimpan data sesuai urutan saat ditambahkan. Oleh karena itu, urutan data yang ditampilkan dapat berbeda dengan urutan penambahan data ke dalam TreeSet.

➤ **Kode program tambahan class TreeSetDemo06.java**

```
System.out.println("First: " + fruits.first());
System.out.println("Last: " + fruits.last());
fruits.remove("Jeruk");
System.out.println("Setelah remove " + fruits);
fruits.pollFirst();
System.out.println("Setelah poll first " + fruits);
fruits.pollLast();
System.out.println("Setelah poll last " + fruits);
```

➤ **Hasil running**

```
Apel
Jambu
Jeruk
Mangga
First: Apel
Last: Mangga
Setelah remove [Apel, Jambu, Mangga]
Setelah poll first [Jambu, Mangga]
Setelah poll last [Jambu]
```

Apa yang dilakukan oleh method first(), last(), remove(), pollFirst(), dan pollLast()?

Jawab:

- first() digunakan untuk mengambil elemen terkecil dalam TreeSet tanpa menghapusnya.
- last() digunakan untuk mengambil elemen terbesar dalam TreeSet tanpa menghapusnya.
- remove() digunakan untuk menghapus elemen tertentu dari TreeSet.
- pollFirst() digunakan untuk mengambil sekaligus menghapus elemen terkecil dalam TreeSet.
- pollLast() digunakan untuk mengambil sekaligus menghapus elemen terbesar dalam TreeSet.

13.6 Praktikum – Sorting

➤ **Kode program class Customer06.java**

```
ArrayList<String> daftarSiswa = new ArrayList<>();
daftarSiswa.add("Zainab");
daftarSiswa.add("Andi");
daftarSiswa.add("Rara");
Collections.sort(daftarSiswa);

System.out.println(daftarSiswa);
customers.sort((c1,c2)->c1.name.compareTo(c2.name));
```

➤ **Hasil running**

```
1
Budi
ID: 1 Nama: Zakia
ID: 5 Nama: Budi Utomo
ID: 100 Nama: Rosa
ID: 4 Nama: Cica
ID: 201 Nama: Della
ID: 202 Nama: Victor
ID: 203 Nama: Sarah
[Andi, Rara, Zainab]
```